

```
$ raghav@portfolio:~ cat README.md
```

Raghav Gupta

Portfolio V1 — Complete Documentation

- 01 Tech Stack — every library, what it does, and why it is here
- 02 Features Built (V1) — file-by-file map of what works
- 03 Placeholder Checklist — everything left to personalize
- 04 Bug-Fix Playbook — diagnose & fix the major features
- 05 Future Roadmap — prioritized upgrades + Claude Code prompts
- 06 Quick Reference — how to edit, restyle, extend & deploy

Table of Contents

Table of Contents	2
Tech Stack	3
Features Built (Version 1)	5
Global & Site-Wide Features	5
Section-by-Section Features	6
Placeholder Content — What To Update	8
How To Fix Common Bugs	12
Future Features Roadmap	15
P0 — Foundation: Do These First	15
P1 — High-Impact Additions	17
P2 — Differentiators	18
P3 — Polish & Personality	19
How To Make Changes — Quick Reference	21
1. Updating Text Content	21
2. Changing Colors & Design Tokens	21
3. Adding a New Section	22
4. Adding a New Project Card	22
5. Deploying Changes To The Live Site	22
6. The /init Command — When To Use It	22

// SECTION 01 — TECH STACK

Tech Stack

Every dependency declared in `package.json`, what it actually does inside this codebase, a beginner-friendly explanation, and a link to the official docs for when you want to go deeper. Versions show the range pinned in `package.json`, with the version actually installed in `node_modules` in parentheses.

Technology	Description	Docs
Next.js ^14.2.5 (14.2.35)	In this project: the core framework — the App Router (everything under <code>app/</code>), font loading via <code>next/font/google</code> for Inter, Space Grotesk and JetBrains Mono, and the <code>dev/build/start</code> scripts. In plain English: Next.js is a toolkit built on top of React that adds page routing, performance optimizations and a production server, so you focus on writing pages instead of plumbing.	nextjs.org/docs
React ^18.3.1 (18.3.1)	In this project: the UI library every <code>.jsx</code> file is written in — components, and hooks like <code>useState</code>, <code>useEffect</code> and <code>useRef</code> power the terminal, theme toggle, filters and animations. In plain English: React lets you build a UI out of small reusable pieces called components that automatically re-render when their data changes.	react.dev
React DOM ^18.3.1 (18.3.1)	In this project: renders the React component tree into the actual browser DOM. Bundled automatically by Next.js — never imported directly in this project. In plain English: the bridge between React's in-memory UI description and the real HTML on the page.	react.dev
Tailwind CSS ^3.4.17 (3.4.19)	In this project: all styling, in every component, via utility classes (<code>flex</code>, <code>text-accent</code>, <code>rounded-xl</code>, etc). Theme colors are mapped to CSS variables in <code>tailwind.config.js</code>. In plain English: instead of writing separate CSS files, you style elements by adding small pre-built class names directly on the HTML/JSX tags.	tailwindcss.com/docs
Framer Motion ^11.18.0 (11.18.2)	In this project: every animation — scroll reveals (<code>whileInView</code>), the Projects filter transitions (<code>AnimatePresence</code> + <code>layout</code>), the Command Palette and Shortcuts modals, the navbar underline, and the page fade in <code>app/template.js</code>. In plain English: a library that makes it easy to animate React components — fade, slide, scale — using simple props instead of hand-written CSS keyframes.	www.framer.com/motion/
next-themes ^0.4.6 (0.4.6)	In this project: dark/light mode. Wraps the whole app in <code>ThemeProvider</code> (<code>app/layout.js</code>, <code>attribute="class"</code>, <code>defaultTheme="dark"</code>) and is read with <code>useTheme()</code> in <code>Navbar</code>, <code>Command Palette</code> and <code>Keyboard Shortcuts</code>. In plain English: a small helper that remembers whether a visitor prefers dark or light mode and applies the matching CSS class to the page.	github.com/pacocoursey/next-themes
Lucide React ^0.468.0 (0.468.0)	In this project: most icons across the site — navbar, hero, about, contact, command palette, shortcuts, achievements, certifications and more. In plain English: a library of clean, consistent SVG icons you use as React components, for example a Mail icon as <code><Mail size={18} /></code>.	lucide.dev

Technology	Description	Docs
React Icons ^5.5.0 (5.6.0)	In this project: brand/tech icons Lucide does not have — the Skills section (Python, React, AWS via the fa6 set, etc.) and the GitHub/LinkedIn icons in Contact and Footer. In plain English: a bundle of many icon packs (Simple Icons, Font Awesome 6, etc.) packaged as React components, for example SiPython or FaGithub.	react-icons.github.io/react-icons
PostCSS ^8.4.49 (8.5.15) devDependency	In this project: processes the CSS pipeline defined in postcss.config.js — this is the engine that actually runs the Tailwind plugin. In plain English: a tool that transforms CSS using small plugins, similar to how Babel transforms JavaScript.	postcss.org
Autoprefixer ^10.4.20 (10.5.0) devDependency	In this project: a PostCSS plugin (configured in postcss.config.js) that automatically adds vendor prefixes to the generated CSS. In plain English: saves you from manually writing browser-specific CSS prefixes like -webkit- or -moz-.	github.com/postcss/autoprefixer

Fonts: three Google Fonts are loaded via next/font/google in app/layout.js — Inter (body text), Space Grotesk (headings/display), and JetBrains Mono (terminal, code, eyebrows). Next.js self-hosts and optimizes them automatically, so there is no separate dependency to track.

// SECTION 02 — FEATURES BUILT (VERSION 1)

Features Built (Version 1)

Everything below is live in the current build, wired up from `app/page.js`. For each feature: the file(s) that implement it, how it behaves in plain English, and where to find it when the site is running locally (`npm run dev` → `http://localhost:3000`).

Global & Site-Wide Features

These apply across the whole site, not to one section.

Feature	File(s)	How it works	Where to find
Theme Toggle	<code>components/Navbar.js</code> <code>app/layout.js</code>	next-themes' <code>ThemeProvider</code> wraps the app in <code>app/layout.js</code> (<code>attribute="class"</code> , <code>defaultTheme="dark"</code>). A button in the navbar calls <code>setTheme()</code> to flip between dark and light, swapping a Sun/Moon icon and toggling the dark class on <code><html></code> so Tailwind's dark: variants apply. The choice is remembered in <code>localStorage</code> .	Top-right of the navbar, on every page.
Sticky Navbar & Active-Section Highlight	<code>components/Navbar.js</code>	A fixed navbar uses an <code>IntersectionObserver</code> to watch every <code><section id="..."></code> . The link matching the section currently in view is highlighted with an animated underline (Framer Motion <code>layoutId</code>).	Top of every page — scroll down and watch the highlighted link change.
Mobile Menu	<code>components/Navbar.js</code>	Below the md breakpoint the link row is replaced by a hamburger icon. Tapping it slides open a full-width dropdown with the same links, animated with <code>AnimatePresence</code> .	Shrink the browser below ~768px (or open on a phone) and tap the menu icon.
Page Transition	<code>app/template.js</code>	Next.js remounts <code>template.js</code> on every navigation. It wraps the page in a <code>motion.div</code> that fades and slides up on mount, giving every route a smooth entrance.	Reload the page or jump between sections — the content fades/slides in.
Custom Cursor	<code>components/CustomCursor.js</code>	Tracks mousemove and renders a small dot plus a larger ring that lerps smoothly toward the pointer every animation frame; the ring grows over links and buttons. Disabled on touch devices and when <code>prefers-reduced-motion</code> is set.	Move the mouse anywhere on a desktop browser — the arrow is replaced by a dot + ring.
Particle Network Background	<code>components/ParticleField.js</code>	A <code><canvas></code> draws up to <code>Math.min(60, (w*h)/22000)</code> particles that drift and connect with faint lines when close, redrawing every frame and resizing with the window. Loaded with <code>next/dynamic</code> so it only runs in the browser.	Behind the text in the Hero section, at the very top of the page.
Command Palette	<code>components/CommandPalette.js</code>	A global keydown listener opens a search modal on <code>Ctrl/Cmd + K</code> . A list of grouped actions — jump to a section, toggle theme, open social links — is filtered live as you type and run on selection.	Press <code>Ctrl+K</code> (⌘K on Mac) from anywhere on the page.

Feature	File(s)	How it works	Where to find
Keyboard Shortcuts Modal	components/KeyboardShortcuts.jsx data/portfolio.js	Pressing ? opens a modal listing every shortcut from the shortcuts array — H/A/S/P/E/C jump to sections, T toggles the theme, Ctrl+K opens the command palette, Esc closes any open modal.	Press ? from anywhere on the page.
SEO & Social Metadata	app/layout.js data/portfolio.js	app/layout.js builds the Next.js metadata object (title, description, Open Graph image, Twitter card) from the seo object in portfolio.js.	View page source (<head>) or paste the site URL into a link-preview tool.

Section-by-Section Features

In page order, top to bottom, exactly as rendered by app/page.js.

Feature	File(s)	How it works	Where to find
Hero #home	components/Hero.jsx components/ParticleField.jsx	Full-viewport intro showing the visitor's name and an animated typewriter (useTypewriterLoop) that types and deletes through heroRoles ("CS Student", "ML Enthusiast", "Problem Solver"). Includes CTA buttons (resume, contact), the particle background, and a bouncing scroll-down chevron.	The very first thing visible when the page loads.
Interactive Terminal #terminal	components/Terminal.jsx	A fake terminal window. When it scrolls into view it auto-types a "boot sequence" (whoami, ls projects/, cat about.txt from terminal.bootSequence), then becomes a live input — typed commands (help, skills, projects, experience, contact, clear, exit...) print the matching response from terminal.responses.	Just below the Hero — click inside it and type "help".
About #about	components/About.jsx	Shows personal.bio and personal.currentlyBuilding, an avatar placeholder with an animated rotating gradient border, and a staggered grid of funFacts (icon + text) that fade/slide in as they enter view.	Scroll past the terminal.
Skills #skills	components/Skills.jsx components/TechTag.jsx	Renders the skills array as category cards (Languages, Frameworks, Tools, AI/ML). Each skill name is mapped to an icon — Lucide or React Icons (e.g. FaAws, SiPytorch) — inside a TechTag pill.	Below About.
Projects #projects	components/Projects.jsx components/ProjectCard.jsx lib/useTilt.js	Filter pills (projectFilters: All/Web/ML/Other) filter the projects array by category; the grid re-renders with AnimatePresence so cards fade/scale in and out. Each ProjectCard tilts in 3D on hover (useTilt), shows tech tags, GitHub/demo links, and a "Coming Soon" badge when comingSoon: true.	Below Skills — click the filter pills to see the transition.
Experience #experience	components/Experience.jsx	Renders the experience array as a vertical timeline, alternating left/right on desktop. Each entry shows company, role, dates, location and bullet points, animated in with whileInView.	Below Projects.
Education #education	components/Education.jsx	Lists each entry from education — school, degree, dates, location, and a detail line — as a simple card list.	Below Experience.

Feature	File(s)	How it works	Where to find
Stats #stats	components/Stats.js lib/useCountUp.js	Displays the four numbers in stats (Projects Built, Internships, GitHub Commits, Years Coding). useCountUp animates each number counting up from 0 once the banner scrolls into view.	Full-width banner below Education.
Certifications #certifications	components/Certifications.jsx	Renders certifications as a grid of cards (name, issuer, year) that link out via link.	Below Stats.
Achievements #achievements	components/Achievements.jsx	Renders achievements as cards, each with an icon (trophy/award/star), a title, and a short description.	Below Certifications.
Contact #contact	components/Contact.jsx	Left side: info cards built from personal (email, phone, location, LinkedIn/GitHub icons). Right side: a name/email/message form that calls e.preventDefault() — it does not send anywhere yet (see Section 4).	Bottom of the main content, before the footer.
Footer	components/Footer.jsx	Site-wide footer with social links, name and copyright line.	Very bottom of every page.

// SECTION 03 — PLACEHOLDER CONTENT

Placeholder Content — What To Update

Every field below lives in `data/portfolio.js` and is currently a placeholder. Nothing else needs to change — edit this one file and the whole site updates. Tick each box off as you replace it with real content.

Personal & Site Info

Field	Current Placeholder Value	What Should Go Here	Priority
☐ <code>personal.github</code>	<code>https://github.com/placeholder-github</code>	Your real GitHub profile URL — used by the Footer, Contact section, and the terminal's contact command.	High
☐ <code>/public/resume.pdf</code>	<code>file does not exist yet</code>	Export your resume as a PDF and save it at <code>/public/resume.pdf</code> (the Hero's "Download Resume" button already points here via <code>personal.resumeUrl</code>).	High
☐ <code>personal.bio</code>	<code>"[PLACEHOLDER] A compelling 3-4 line bio goes here..."</code>	3-4 sentences in your own voice: who you are, what you build, what you're learning right now, and what kind of problems excite you. Shown in the About section.	High
☐ <code>personal.currently Building</code>	<code>"[PLACEHOLDER] An ML-powered something..."</code>	One line about your current side project, or delete this line from <code>About.jsx</code> if you don't want to show it.	Medium
☐ <code>funFacts[0].text</code>	<code>"Fueled by an alarming amount of chai"</code>	A real, light/funny personal fact.	Low
☐ <code>funFacts[1].text</code>	<code>"Vim keybindings everywhere, no regrets"</code>	A real fact about your tools or workflow.	Low
☐ <code>funFacts[2].text</code>	<code>"...places you have lived or studied"</code>	Something about where you're from, or have lived/studied.	Low
☐ <code>funFacts[3].text</code>	<code>"Hackathon regular, sleep optional"</code>	A real fact about hackathons, competitions, or habits.	Low
☐ <code>seo.description</code>	<code>"[PLACEHOLDER] Portfolio of Raghav Gupta..."</code>	A real ~150-160 character summary of you and the site, used by search engines and link previews.	High
☐ <code>seo.url</code>	<code>https://placeholder-domain.com</code>	The real deployed URL (Vercel default or custom domain) — used for canonical and Open Graph tags.	High
☐ <code>seo.ogImage</code>	<code>/images/og-image.png (missing)</code>	A real 1200×630 social-preview image saved at <code>/public/images/og-image.png</code> .	Medium
☐ <code>seo.twitterHandle</code>	<code>@placeholder</code>	Your real X/Twitter handle, or remove the twitter block from <code>app/layout.js</code> if you don't have one.	Low

Projects

Field	Current Placeholder Value	What Should Go Here	Priority
<code>projects[0]</code> – Project Alpha	<i>name, description, github & demo URLs all placeholders</i>	Real project name, a 1-2 line description of what it does and why it's impressive, the real GitHub repo URL, and a real demo URL (or remove the demo key/button if there isn't one).	High
<code>projects[1]</code> – Project Beta	<i>name, description, github & demo URLs all placeholders</i>	Same as Project Alpha — real name, description, GitHub link, and demo link.	High
<code>projects[2]</code> – Project Gamma	<i>name, description, github & demo URLs all placeholders</i>	Same as Project Alpha — real name, description, GitHub link, and demo link.	High
<code>projects[3]</code> – Project Delta	<i>placeholders, comingSoon: true</i>	Real name, description and links once it's ready. Keep <code>comingSoon: true</code> (shows a badge instead of links) until then, or set it to <code>false</code> when you ship it.	Medium

Experience

All three entries in experience are fully placeholder. If you don't have three real internships/jobs yet, delete the extra entries from the array (and consider removing the Experience section from app/page.js entirely until you do) rather than shipping placeholder text.

Field	Current Placeholder Value	What Should Go Here	Priority
☐ experience[0] – Company One	<i>company, role, dates, location & 3 bullets all placeholders</i>	Real company/organization name, your role or title, start-end dates, location, and three bullet points: what you built/shipped, a metric or impact statement, and the tech you used.	High
☐ experience[1] – Company Two	<i>company, role, dates, location & 3 bullets all placeholders</i>	Same structure as Company One — or delete this entry if you only have one experience to list.	High
☐ experience[2] – Company Three	<i>company, role, dates, location & 3 bullets all placeholders</i>	Same structure as Company One — or delete this entry if you have fewer than three.	Medium

Education

Field	Current Placeholder Value	What Should Go Here	Priority
☐ education[0] – University One	<i>school, degree, dates, location, detail all placeholders</i>	Your current/most recent school, exact degree & field of study, dates, location, and a detail line (focus areas, GPA, relevant coursework).	High
☐ education[1] – University Two	<i>school, degree, dates, location, detail all placeholders</i>	Your previous school (e.g. high school), or delete this entry if you only want one listed.	High

Certifications

Field	Current Placeholder Value	What Should Go Here	Priority
☐ certifications[0]	<i>"Certification One" · Issuer Name · 2025 · example.com/cert-1</i>	Real certificate name, issuing platform (Coursera, AWS, etc.), year earned, and a working link to view/verify it.	Medium
☐ certifications[1]	<i>"Certification Two" · Issuer Name · 2024 · example.com/cert-2</i>	Same as above, or delete this entry if you have fewer than four certifications.	Medium
☐ certifications[2]	<i>"Certification Three" · Issuer Name · 2024 · example.com/cert-3</i>	Same as above, or delete this entry if you have fewer than four certifications.	Medium
☐ certifications[3]	<i>"Certification Four" · Issuer Name · 2023 · example.com/cert-4</i>	Same as above, or delete this entry if you have fewer than four certifications.	Low

Achievements

Field	Current Placeholder Value	What Should Go Here	Priority
<code>achievements[0]</code>	"Achievement One" + description (icon: trophy)	A real award, competition placement, publication or notable accomplishment, with a one-line description and context.	Medium
<code>achievements[1]</code>	"Achievement Two" + description (icon: award)	Same as above, or delete this entry if you have fewer than three achievements.	Medium
<code>achievements[2]</code>	"Achievement Three" + description (icon: star)	Same as above, or delete this entry if you have fewer than three achievements.	Low

Stats

All four numbers in stats are placeholders — the comment above the array literally says so. Update them to real, defensible numbers.

Field	Current Placeholder Value	What Should Go Here	Priority
<code>stats[0]</code> – "Projects Built"	12	Your real count of finished/in-progress projects worth showing.	Medium
<code>stats[1]</code> – "Internships"	3	Your real internship/work-experience count (should match the experience array).	Medium
<code>stats[2]</code> – "GitHub Commits"	500+	Your real contribution count — check your GitHub profile's contribution graph.	Medium
<code>stats[3]</code> – "Years Coding"	2	How long you've actually been coding.	Low

Terminal Responses

These live in `terminal.responses` and are printed when a visitor types the matching command into the interactive terminal. Keep them short and consistent with the sections above.

Field	Current Placeholder Value	What Should Go Here	Priority
<code>terminal.responses.whoami[1]</code>	"Delhi, India · [PLACEHOLDER] one-liner about you"	A real one-line description of yourself, consistent with <code>personal.bio</code> .	Medium
<code>terminal.responses["cat about.txt"]</code>	3 placeholder lines	A short About blurb that mirrors <code>personal.bio</code> , split across 2-3 lines.	Medium
<code>terminal.responses.projects</code>	4 placeholder one-liners	One short line per project (project-alpha..delta) summarizing what each one does, matching the <code>projects</code> array.	Medium
<code>terminal.responses.experience</code>	3 placeholder lines	One line per real experience entry ("Company · Role · Dates"), or a single line saying you're open to opportunities if the array is empty.	Medium

// SECTION 04 — BUG-FIX PLAYBOOK

How To Fix Common Bugs

A diagnostic checklist for the seven most interactive features. For each: what tends to break, how to confirm that's actually the problem, the file to open, and a ready-to-paste prompt for Claude Code.

1. Terminal

What can go wrong: The boot sequence (`whoami`, `ls projects/`, `cat about.txt`) never types out, typing freezes partway, a command you type does nothing, or `clear` doesn't clear the screen.

How to identify: Open the browser console (F12) for red errors. Then check `data/portfolio.js` — is the command you typed an exact-match key in `terminal.responses`? Commands are case-sensitive and must match exactly (e.g. `ls projects/` including the slash).

Files to check: `components/Terminal.jsx`, `data/portfolio.js` (terminal)

Tell Claude Code:

```
The terminal in components/Terminal.jsx isn't typing out the boot sequence on load /
isn't responding to the "<command>" command. Debug the typing effect and the
runCommand handler and fix it.
```

2. Command Palette

What can go wrong: `Ctrl+K` / `Cmd+K` doesn't open the palette, the search box doesn't filter results, clicking an item doesn't navigate or close the modal, or `Esc` doesn't close it.

How to identify: Try both `Ctrl+K` (Windows/Linux) and `Cmd+K` (Mac) — some browsers reserve one of these for a built-in shortcut. Check the console for errors when the palette is open and you type or click.

Files to check: `components/CommandPalette.jsx`

Tell Claude Code:

```
Ctrl+K (and Cmd+K on Mac) isn't opening the Command Palette in
components/CommandPalette.jsx — check the global keydown listener and the
key-combination check (e.target, metaKey/ctrlKey) and fix it.
```

3. Animations

What can go wrong: A section never fades/slides in (stuck invisible at opacity 0), an animation replays every time you scroll past it instead of once, animations feel janky on scroll, or motion doesn't stop for visitors with reduced-motion enabled.

How to identify: In React DevTools, check the element's computed opacity — if it's stuck at 0, the `whileInView/viewport` condition in `lib/animations.js` or the component never matched. For replay issues, check whether `viewport={{ once: true }}` is set. For reduced motion, check the media query in `app/globals.css`.

Files to check: `lib/animations.js`, the affected `components/*.jsx`, `app/globals.css`

Tell Claude Code:

```
The <Section name> section never animates in / re-animates every time I scroll
past it. Check the Framer Motion variants and viewport settings in
components/<Component>.jsx and lib/animations.js and fix the trigger so it
animates in once, respecting prefers-reduced-motion.
```

4. Dark / Light Toggle

What can go wrong: Clicking the toggle does nothing, the page flashes the wrong theme for a moment on load (FOUC), the icon doesn't swap between sun/moon, or some elements keep their color regardless of theme.

How to identify: Open devtools and watch the `<html>` element's `class` attribute while toggling — it should flip between dark and nothing. If it doesn't change, the bug is in `Navbar.jsx`'s click handler or `useTheme()`. If the class flips but colors don't, the affected element is using a hardcoded color instead of a `dark:-aware` Tailwind class or CSS variable.

Files to check: `app/layout.js` (`ThemeProvider`), `tailwind.config.js` (`darkMode`), `components/Navbar.jsx`

Tell Claude Code:

```
Toggling the theme button in the navbar doesn't change the page colors. Check that ThemeProvider in app/layout.js has attribute="class", that tailwind.config.js has darkMode: 'class', and that Navbar.jsx's toggle button correctly calls setTheme() from next-themes. Fix whichever part is broken.
```

5. Contact Form

What can go wrong: You fill in the form and click Send — and nothing happens. **This is expected in V1**, not a bug: `components/Contact.jsx` calls `e.preventDefault()` with a `// Form integration coming soon` comment and never sends the data anywhere.

How to identify: Open the console and submit the form — there will be no network request in the Network tab, by design.

Files to check: `components/Contact.jsx`

Tell Claude Code:

This isn't something to "fix" in isolation — it's the single highest-impact item in the roadmap. See **Section 5 – "Real Contact Form with EmailJS"** for the full prompt to wire this up properly.

6. Particles

What can go wrong: The particle background in the Hero doesn't render at all, it lags or drops frames (especially on laptops/phones), or it visually overflows its container.

How to identify: If nothing renders: check the console for canvas errors, and confirm `ParticleField` is still loaded via `next/dynamic` with server-side rendering disabled in `Hero.jsx`. If it lags: open the Performance tab and look for a long frame time inside the `requestAnimationFrame` loop — likely too many particles for the screen size.

Files to check: `components/ParticleField.jsx`, `components/Hero.jsx`

Tell Claude Code:

```
The particle background in components/ParticleField.jsx is laggy on my laptop. Reduce the max particle count (currently Math.min(60, (w*h)/22000)) and/or the distance threshold used to draw connecting lines so it runs smoothly on lower-end devices.
```

7. Navbar

What can go wrong: The highlighted nav link never changes (or highlights the wrong section) while scrolling, the mobile menu stays open after you tap a link, or clicking a link scrolls to a section whose top is hidden behind the fixed navbar.

How to identify: For the highlight bug, check the `IntersectionObserver` options (`threshold/rootMargin`) in `Navbar.jsx` and confirm every section element's `id` matches the `hrefs` in `navLinks`. For the overlap bug, compare the navbar's rendered height to each section's `scroll-margin-top` in `app/globals.css`.

Files to check: `components/Navbar.jsx`, `app/globals.css`, `data/portfolio.js` (`navLinks`)

Tell Claude Code:

```
Clicking a navbar link scrolls to the right section, but its top is hidden behind
the fixed navbar / the active-section highlight in Navbar.jsx doesn't track
scrolling correctly. Fix the scroll-margin-top / IntersectionObserver settings so
both work correctly.
```

// SECTION 05 — FUTURE ROADMAP

Future Features Roadmap

19 upgrades, grouped by priority, that would take this from a solid V1 to a world-class CS student portfolio. Each one lists why it matters, how hard it is, roughly how long it takes with Claude Code, and an exact prompt you can paste in to build it.

P0 — Foundation: Do These First

Nothing else on this list matters until these are done — they turn this from a demo into a real, live portfolio with your real information.

1. Replace All Placeholder Content

Why it helps: This is the single highest-leverage change you can make. Right now every project, experience entry, school, certification, achievement and bio is marked [PLACEHOLDER]. No new feature matters if a recruiter's first impression is fake content. Section 3 of this document is your checklist.

Difficulty: **Easy** **Time with Claude Code:** 2-4 hours (writing, not coding)

Prompt for Claude Code:

```
I've updated data/portfolio.js with my real bio, projects, experience, education, certifications, achievements and stats, replacing the [PLACEHOLDER] values. Please review the file for any [PLACEHOLDER] markers I missed, check that array lengths still match what each component expects, and flag anything that looks inconsistent or broken.
```

2. Add Real Resume, Photo & OG Image

Why it helps: The Hero's "Download Resume" button and the About section's avatar currently point at files that don't exist. A missing resume link or a placeholder avatar looks unfinished. A real Open Graph image also makes links you share on LinkedIn/X/WhatsApp look professional instead of blank.

Difficulty: **Easy** **Time with Claude Code:** 30-60 min

Prompt for Claude Code:

```
I've added /public/resume.pdf, /public/images/avatar.jpg and /public/images/og-image.png (1200x630). Update components/About.jsx to show the real avatar image instead of the placeholder gradient box, confirm Hero.jsx's resume button links to /resume.pdf, and update seo.ogImage in data/portfolio.js to point at the new OG image.
```

3. Deploy to Vercel + Custom Domain

Why it helps: A portfolio that only runs on localhost doesn't exist to anyone else. Vercel is free for personal projects, deploys straight from a GitHub repo, and gives you the real URL needed for seo.url, your resume, and your LinkedIn/GitHub profiles.

Difficulty: **Easy** **Time with Claude Code:** 15-30 min (mostly the Vercel dashboard, not Claude Code)

Prompt for Claude Code:

```
Before I deploy this Next.js app to Vercel, run npm run build and fix any build errors or warnings. Double-check next.config.mjs for anything that needs to change for production (e.g. remote image domains), and confirm there's nothing in the repo that shouldn't be public (API keys, .env files).
```

4. Real Contact Form with EmailJS

Why it helps: Right now the contact form does nothing when submitted. For a CS student portfolio, a working contact form is one of the first things visitors test — a broken one undermines everything else on the page. EmailJS sends real email from a static frontend with no backend server needed.

Difficulty: **Medium** **Time with Claude Code:** 1-2 hours

Prompt for Claude Code:

```
Wire up the contact form in components/Contact.jsx using EmailJS (@emailjs/browser). On submit, send the name/email/message fields to my email via an EmailJS service + template (service ID, template ID and public key will be in env vars NEXT_PUBLIC_EMAILJS_SERVICE_ID, _TEMPLATE_ID and _PUBLIC_KEY). Add loading, success and error states (disable the button while sending, show a success message, show an error message on failure), keeping the existing styling and animations.
```


P1 — High-Impact Additions

Once the foundation is real, these are the upgrades that most directly make the portfolio look more credible and more discoverable.

5. GitHub Activity Graph

Why it helps: Shows real, continuously-updating proof of how active you are as a developer — something recruiters actually look at. It's also a "living" element that keeps the page feeling current without you editing anything.

Difficulty: **Easy** **Time with Claude Code:** 30-60 min

Prompt for Claude Code:

```
Add a GitHub contribution activity graph to the About section (or a new section) using an image-based service such as https://ghchart.rshah.org/<username>, wrapped in a styled card that matches the site's existing dark/light theme. Use my real GitHub username from personal.github in data/portfolio.js.
```

6. Project Case Study Pages

Why it helps: A card with a two-line description and two links is the bare minimum. A full case-study page (problem, approach, architecture, challenges, results, what you'd do differently) is what separates a "list of projects" from a portfolio that gets you hired — it shows how you think.

Difficulty: **Hard** **Time with Claude Code:** 4-8 hours (1 template + writing per project)

Prompt for Claude Code:

```
Create a dynamic route app/projects/[slug]/page.js that renders a full case-study page for each project in data/portfolio.js. Add a slug and a caseStudy object (problem, approach, techDecisions, challenges, results, screenshots) to each project. Make each ProjectCard's 'View Details' link to /projects/[slug], and add a back-to-portfolio link with styling that matches the rest of the site in both dark and light mode.
```

7. Vercel Analytics

Why it helps: Free, privacy-friendly visitor analytics with almost zero setup. Lets you see which sections people scroll to, which projects get clicked, and how many visitors you're actually getting — useful talking points and a sign of basic product thinking.

Difficulty: **Easy** **Time with Claude Code:** 10-15 min

Prompt for Claude Code:

```
Install @vercel/analytics, import the Analytics component, and add it to app/layout.js so visits are tracked once this is deployed on Vercel.
```

8. SEO: JSON-LD, Sitemap & robots.txt

Why it helps: Structured data (a JSON-LD Person schema) helps Google understand who you are and can produce a rich snippet in search results. A sitemap and robots.txt are basic SEO hygiene that take minutes and make the whole site fully crawlable.

Difficulty: **Easy** **Time with Claude Code:** 30-45 min

Prompt for Claude Code:

```
Add a JSON-LD <script type="application/ld+json"> block to app/layout.js describing me as a schema.org Person (name, jobTitle, url, sameAs: [linkedin, github] — pull these from data/portfolio.js). Also create app/sitemap.js and app/robots.js using Next.js's built-in metadata file conventions, pointing at seo.url.
```

P2 — Differentiators

Features most other student portfolios don't have — these are what make someone remember the site after they close the tab.

9. AI Chatbot About Me (Anthropic API)

Why it helps: A genuinely novel feature for a CS portfolio — a chatbot, grounded in your resume and projects, that recruiters can ask questions like "What's Raghav's strongest project?" or "Does he know Docker?". It also doubles as a live demo of you building with the Anthropic API.

Difficulty: **Hard** **Time with Claude Code:** 4-6 hours

Prompt for Claude Code:

```
Add an 'Ask about me' chat widget: a floating button (bottom-right) that opens a chat panel. Create a Next.js Route Handler at app/api/chat/route.js that calls the Anthropic API (model claude-haiku-4-5, key from an ANTHROPIC_API_KEY env var) with a system prompt built from my data/portfolio.js content (bio, skills, projects, experience), so it can answer questions about me. Build a ChatWidget component with a message list, input box, loading state, and the site's existing dark/light styling and animation conventions.
```

10. /uses Page

Why it helps: A classic personal-site page (in the style of uses.tech) listing your hardware, editor, terminal setup and favorite tools. It's low effort, very "developer culture", and gives visitors a sense of your day-to-day setup and personality.

Difficulty: **Easy** **Time with Claude Code:** 1-2 hours

Prompt for Claude Code:

```
Create a new page at app/uses/page.js styled to match the rest of the site (same fonts, colors, terminal-style headers). Add a usesData export to data/portfolio.js with categories like Hardware, Editor & Terminal, Languages & Frameworks, and Other Tools, each an array of { name, description }. Render them as a grid of cards, and add a link to /uses from the Footer.
```

11. Blog Section

Why it helps: Writing about what you build — a project postmortem, "what I learned this month", a short tutorial — demonstrates communication skills and gives Google more pages to index. Both directly help with internship and job applications.

Difficulty: **Hard** **Time with Claude Code:** 4-8 hours (MDX setup + first posts)

Prompt for Claude Code:

```
Set up a blog at /blog using MDX (@next/mdx) — app/blog/page.js listing posts, and app/blog/[slug]/page.js rendering individual .mdx files from a content/blog/ directory (frontmatter: title, date, summary, tags). Style the list and post pages to match the site's existing fonts/colors/dark mode, and add 'Blog' to navLinks in data/portfolio.js.
```

12. Visitor Counter

Why it helps: A small, fun "X people have visited this page" counter is a nostalgic personal-site touch that's easy to add with a free service and gives the page a sense of life.

Difficulty: **Easy** **Time with Claude Code:** 30-45 min

Prompt for Claude Code:

```
Add a small visitor counter to the Footer using a free hit-counter badge service (e.g. an <img> badge from https://hits.sh), styled to fit the site's dark/light footer.
```

13. Project Media (Screenshots / GIFs / Demo Videos)

Why it helps: Right now ProjectCard is text-only. A screenshot, GIF, or short demo video of each project is the single biggest visual upgrade to the Projects section — people judge a project by what it looks like before they read a word about it.

Difficulty: **Medium** **Time with Claude Code:** 1-3 hours (depends on capturing media)

Prompt for Claude Code:

```
Add an optional media field (image or short looping video) to each project in data/portfolio.js, and update components/ProjectCard.jsx to show it at the top of the card (16:9, rounded corners, lazy-loaded with next/image), falling back to the current text-only layout if media is missing.
```

14. Calendly Booking Embed

Why it helps: Lets recruiters or collaborators book a quick call directly from the site — removes the back-and-forth of scheduling emails and signals that you're open to conversations.

Difficulty: **Easy** **Time with Claude Code:** 30-45 min

Prompt for Claude Code:

```
Add a 'Book a call' button to the Contact section that opens a Calendly popup widget (react-calendly), using a Calendly URL stored as personal.calendly in data/portfolio.js. Style the button to match the existing Contact section buttons.
```

P3 — Polish & Personality

Smaller touches that round out the experience and reward visitors who explore.

15. Lighthouse & Accessibility Audit

Why it helps: Performance, accessibility and SEO scores are concrete numbers you can put on a resume ("100/100 Lighthouse score"). The audit also catches real issues — missing alt text, low-contrast text, unlabeled icon buttons.

Difficulty: **Easy** **Time with Claude Code:** 1-2 hours

Prompt for Claude Code:

```
Run a Lighthouse audit on the deployed site (or locally via npm run build && npm run start) and fix the top issues it reports. Focus on accessibility (alt text, color contrast, aria-label on icon-only buttons like the theme toggle and command palette) and performance (image sizes, unused CSS, font loading).
```

16. Terminal Easter Eggs

Why it helps: The interactive terminal is already the most distinctive feature on the site. A couple of hidden or fun commands rewards visitors who explore and makes the site memorable.

Difficulty: **Easy** **Time with Claude Code:** 1 hour

Prompt for Claude Code:

```
Add a few easter-egg commands to terminal.responses in data/portfolio.js and components/Terminal.jsx — for example 'sudo' (a joke 'permission denied' response), 'coffee' (small ASCII art), and 'matrix' (a brief animated effect). Keep them light and on-brand, and list them in 'help' only if you want them discoverable.
```

17. Konami Code Easter Egg

Why it helps: A site-wide easter egg (↑↑↓↓←→←→BA triggers a fun visual effect) is a tiny, well-known nod to developer/gamer culture — the kind of detail that gets noticed and shared.

Difficulty: **Easy** **Time with Claude Code:** 1 hour

Prompt for Claude Code:

```
Add a global Konami code listener (up up down down left right left right b a) that, when triggered, plays a fun full-screen effect (e.g. a confetti burst or brief CSS animation) and shows a small toast message. Implement it as a component included once in app/page.js, and skip the effect entirely when prefers-reduced-motion is set.
```

18. Testimonials / Recommendations

Why it helps: A short quote from a professor, mentor, or teammate adds third-party credibility that's hard to fake — even one or two genuine quotes make a portfolio feel more trustworthy.

Difficulty: **Medium** **Time with Claude Code:** 1-2 hours

Prompt for Claude Code:

```
Add a Testimonials section between Experience and Education. Add a testimonials array to data/portfolio.js (each with quote, name, role, and optional avatar), create components/Testimonials.jsx rendering them as a card carousel or grid using the site's existing animation and styling conventions, and add it to app/page.js.
```

19. Micro-Interaction Polish Pass

Why it helps: A final pass over small details — hover states, focus rings for keyboard users, button feedback, loading skeletons — is what makes a site feel "finished" rather than "a tutorial project". These are details reviewers notice even if they can't say why a site feels polished.

Difficulty: **Medium** **Time with Claude Code:** 2-4 hours

Prompt for Claude Code:

```
Do a polish pass across the site: ensure every interactive element (buttons, links, filter pills, theme toggle, command palette items) has a visible focus-visible ring for keyboard navigation, add subtle hover/active scale or color transitions where missing, and check that all icon-only buttons have aria-label attributes.
```

// SECTION 06 — QUICK REFERENCE

How To Make Changes — Quick Reference

A cheat-sheet for the everyday edits — what to touch, what to leave alone, and how to get a change live.

1. Updating Text Content

Every piece of visible text — name, bio, projects, skills, experience, education, certifications, achievements, stats, terminal responses, nav links, SEO copy — lives in one file: `data/portfolio.js`.

To change something: open `data/portfolio.js`, find the relevant export (e.g. `personal`, `projects`, `experience`), edit the value, and save. The dev server (`npm run dev`) hot-reloads instantly.

Don't edit visible text directly inside component `.jsx` files — components only render whatever `portfolio.js` gives them, so text changes belong in one place.

2. Changing Colors & Design Tokens

All colors are CSS variables defined in `app/globals.css`, in two blocks: `:root`, `.dark` (dark theme — the default) and `.light` (light theme). They are mapped to Tailwind class names in `tailwind.config.js` (`theme.extend.colors`), so classes like `bg-bg-primary` or `text-accent` work everywhere.

Token	Dark Value	Light Value	Used For
<code>--bg-primary</code>	<code>#0d1117</code>	<code>#ffffff</code>	Page background
<code>--bg-secondary</code>	<code>#161b22</code>	<code>#f6f8fa</code>	Card / section backgrounds
<code>--bg-tertiary</code>	<code>#21262d</code>	<code>#eaeef2</code>	Borders, dividers, hover backgrounds
<code>--accent</code> <code>--text-accent</code>	<code>#00f5ff</code>	<code>#0969da</code>	Accent color — links, highlights, the cyan glow, active nav underline
<code>--accent-dim</code>	<code>rgba(0,245,255,.13)</code>	<code>rgba(9,105,218,.12)</code>	Soft glow / shadow tint (<code>boxShadow.glow</code>)
<code>--text-primary</code>	<code>#e6edf3</code>	<code>#1f2328</code>	Main body text
<code>--text-secondary</code>	<code>#8b949e</code>	<code>#656d76</code>	Muted / secondary text
<code>--success</code>	<code>#3fb950</code>	<code>#1a7f37</code>	Success states
<code>--warning</code>	<code>#d29922</code>	<code>#9a6700</code>	Warning states

To re-theme the site: change the values in both the `.dark` and `.light` blocks in `app/globals.css` — every component already uses these variables via Tailwind, so a single edit updates the whole site. Fonts (Inter, Space Grotesk, JetBrains Mono) are configured in `app/layout.js` and mapped via `fontFamily` in `tailwind.config.js`.

3. Adding a New Section

- 1 Create `components/NewSection.jsx` — copy the structure of a simple existing section (e.g. `Education.jsx`) for the `<section id="...">` wrapper, `SectionHeader`, and animation pattern.
- 2 Add any content for it as a new export in `data/portfolio.js` (an array or object, following the style of existing exports like achievements).
- 3 Import the component in `app/page.js` and place it inside `<main>` where you want it to appear.
- 4 If it should appear in the navbar, add `{ label, href: '#your-id' }` to `navLinks` in `data/portfolio.js`.
- 5 If it should get a keyboard shortcut, add an entry to `shortcuts` in `data/portfolio.js` and to the `sectionKeys` map in `KeyboardShortcuts.jsx`.
- 6 Give the section a `scroll-margin-top` matching the navbar height in `app/globals.css`, and animate it in with `fadeInUp / whileInView` from `lib/animations.js` to match the rest of the site.

4. Adding a New Project Card

- 1 Open `data/portfolio.js` and find the `projects` array.
- 2 Copy an existing project object (e.g. `Project Delta`) as a template.
- 3 Fill in `name`, `description`, `tech` (array of tag strings), `category` (must match a value in `projectFilters` — `web`, `ml`, or `other` — or add a new filter to that array), `github`, `demo`, and `comingSoon` (true to show a "Coming Soon" badge instead of links).
- 4 Save — the new card appears automatically in the Projects grid and is included in whichever filter matches its category.
- 5 If a tech tag needs an icon that doesn't exist yet, add it to the `iconMap` in `Skills.jsx / TechTag.jsx` — Lucide first, React Icons (fa6/si sets) as a fallback.

5. Deploying Changes To The Live Site

This project is built to deploy on Vercel (made by the team behind Next.js, with a generous free tier for personal sites).

- 1 Commit your changes and push to the GitHub repository connected to the Vercel project.
- 2 Vercel automatically builds and deploys on push — usually live within 1-2 minutes. Check the Vercel dashboard for build logs if something fails.
- 3 To test a production build locally first, run `npm run build` then `npm run start`.
- 4 Pushing to a branch or opening a pull request gets its own preview URL automatically — useful for trying bigger changes before merging to main.

Not deployed yet? See [Section 5, item 3](#) — "Deploy to Vercel + Custom Domain".

6. The `/init` Command — When To Use It

`/init` is a built-in Claude Code slash command that scans the project and writes (or updates) `CLAUDE.md` — a memory file Claude Code automatically reads at the start of every session in this folder, describing the codebase structure, conventions, and useful commands.

Run it when:

- It's your first time using Claude Code in this project and there's no `CLAUDE.md` yet.
- You make a major structural change — new top-level folders, a new framework or tool, a big refactor — so Claude Code's understanding stays accurate.
- Claude Code keeps getting confused about where things live or which commands to run.

It's safe to run anytime — it only (re)writes `CLAUDE.md`. Review the diff afterward and tweak it by hand if anything looks off.

Appendix — Terminal Update (June 27, 2026)

Supersedes the Interactive Terminal sections on pages 6, 11, and 12.

Overview

The terminal now behaves like a fake Unix filesystem. Commands are defined as exact-match keys in `data/portfolio.js` → `terminal.responses`. The boot sequence is unchanged: `whoami`, `ls projects/`, `cat about.txt`.

Available Commands (type `help`)

<code>help</code>	show available commands
<code>whoami</code>	who is this guy?
<code>pwd</code>	print working directory → <code>/home/raghav/portfolio</code>
<code>ls</code>	list directory contents (one entry per line)
<code>ls projects/</code>	list project directories (one folder per line)
<code>tree</code>	display directory tree (includes <code>projects/</code> subtree)
<code>cat about.txt</code>	display about information
<code>cat skills.txt</code>	display technical skills
<code>cat experience.txt</code>	display work experience
<code>cat contact.txt</code>	display contact information
<code>cat projects.txt</code>	display project summaries
<code>clear</code>	clear the terminal
<code>exit</code>	exit terminal

Removed Commands

These no longer work (return: `bash: <cmd>: command not found`):

- `skills`
- `projects`
- `experience`
- `contact`

Fake Filesystem

`ls` lists: `about.txt`, `skills.txt`, `experience.txt`, `contact.txt`, `projects.txt`, `projects/`
`tree` expands `projects/` with `project-alpha/ ... project-delta/`
`cat projects.txt` shows detailed project lines; `ls projects/` shows folder names only — same split as a real shell.

Implementation

Files: `data/portfolio.js` (commands + output), `components/terminal.jsx`
(`runCommand` lookup; unknown commands → `bash: <cmd>: command not found`)
No autocomplete, command history, or filesystem navigation logic.

Appendix — AI Chatbot Update (July 8, 2026)

Implements (with changes) roadmap item 9, 'AI Chatbot About Me', from Section 5 (page 18). That roadmap entry suggested the Anthropic API; the shipped version uses OpenAI instead.

Overview

A floating 'ask-ai' button in the bottom-right corner opens a terminal-styled chat panel (same macOS title bar, JetBrains Mono, and prompt styling as the Terminal section). Visitors can hold a real conversation with an assistant grounded in the site's own content: it answers questions about Raghav's bio, skills, projects, experience, education, certifications, achievements and stats, handles reasonable general questions, politely declines anything inappropriate for a portfolio, and never invents facts — unknowns are redirected to the Contact section. Message history lives only in component state (nothing is persisted).

New Files

<code>app/api/chat/route.js</code>	Server-side Route Handler. Builds a context string from every export in <code>data/portfolio.js</code> (full-content injection into the system prompt – no embeddings/RAG; the site is small enough), then calls OpenAI (model <code>gpt-4o-mini</code>) with the API key read from <code>process.env.OPENAI_API_KEY</code> . The key never leaves the server.
<code>components/ChatWidget.jsx</code>	Client widget: launcher button + chat panel, message list, loading indicator ('thinking' + blinking cursor), and visible error lines when a request fails. Lazy-loaded in <code>app/page.js</code> via <code>dynamic(..., { ssr: false })</code> , like Terminal and CommandPalette.
<code>.env.example</code>	Committed template listing <code>OPENAI_API_KEY=</code> (empty).
<code>.env.local</code>	NOT committed (gitignored) – holds the real key locally.

Environment Variable — Required Setup

Anyone cloning the repo must create `.env.local` before the chatbot works:

```
cp .env.example .env.local # then paste your OpenAI key after OPENAI_API_KEY=
```

Without the key the site still builds and runs; the chat endpoint returns a friendly 'not configured' message (HTTP 503). When deploying to Vercel (Section 6, item 5), add `OPENAI_API_KEY` under Project Settings > Environment Variables.

`.gitignore` keeps ignoring every `.env*` file but now explicitly un-ignores `.env.example`.

Abuse Guards & Error Handling

Client: input capped at 500 characters, minimum 1.2s between sends, send disabled while a reply is pending, only the last 12 turns are sent per request.

Server: message shape and length validated (max 1000 chars), per-IP in-memory rate limit (10 requests/minute, HTTP 429), replies capped at 400 tokens, OpenAI errors mapped to friendly JSON messages — nothing fails silently or leaks provider error details.

Placement & Site Conventions

Fixed bottom-right at z-index 45: under the mobile menu (50), Command Palette (60) and custom cursor (100), so nothing is blocked; Ctrl+K still opens the palette above the chat. Esc closes the panel like other overlays. Open/close animation is skipped when `prefers-reduced-motion` is set (`framer-motion useReducedMotion`); remaining chrome uses color-only hover transitions, safe on touch devices. Both themes use the existing CSS variable tokens. No new npm dependencies were added — Section 1 (Tech Stack) is unchanged.

Known Limitation

`data/portfolio.js` still contains [PLACEHOLDER] content (see Section 3). The grounding is injected as-is, but the system prompt instructs the assistant to treat placeholder-marked fields as unpublished rather than repeat them. Filling in real content (Section 3 checklist)

automatically improves the chatbot's answers — no code changes needed.